DocuChat

Chat with your PDFs and get answers with exact page citations

A Retrieval-Augmented Generation (RAG) project report

At a glance
DocuChat lets you upload PDF documents and ask questions in plain language. Every answer is grounded in your documents and cites the exact page it came from, so you can verify it instead of just trusting the AI.

1. Introduction and Problem Statement

Reading long PDFs such as research papers, textbooks, manuals, and contracts to find a single answer is slow. General AI chatbots can summarize, but they often invent details and cannot show where an answer came from. DocuChat solves this by answering only from the user's own documents and citing the page for every claim.

2. Objectives

- Allow users to upload one or more PDF files and query them in natural language.
- Return accurate answers backed by page-level citations.
- Avoid guessing: reply that the answer was not found when the documents do not contain it.
- Keep the design simple, modular, and able to run locally for private documents.

3. Key Features

Feature	Description
PDF upload	Upload one or many PDF files at once.
Natural language Q&A	Ask questions the way you would ask a person.
Page citations	Each answer shows file name and page number, for example report.pdf, p. 12.
Semantic search	Finds passages by meaning, not just matching keywords.
Honest fallback	Says it could not find the answer rather than hallucinating.
Multi-document support	Search across several documents in one session.
Local option	Can run fully offline with local embeddings and a local model.

4. System Architecture and Workflow

DocuChat has two phases: **ingestion** (done once per document) and **question answering** (done for every question).

Phase A: Ingestion

- **Extract** text from each PDF page while keeping the page number.
- **Chunk** the text into small overlapping pieces (for example 500 to 1000 characters).
- **Embed** each chunk into a vector that represents its meaning.
- **Store** vectors and metadata (file name, page number) in a vector database.

Phase B: Question Answering

- **Embed** the user's question with the same embedding model.
- **Retrieve** the top-k most similar chunks from the vector database.
- **Prompt** the LLM with the question and the retrieved chunks, instructing it to answer only from them.
- **Return** the answer together with the page citations taken from chunk metadata.

Workflow Diagram

```
PDF files
   |
   v
[1] Extract text page by page (keep page numbers)
   |
   v
[2] Split into overlapping chunks
   |
   v
[3] Convert chunks to embeddings
   |
   v
[4] Store in vector database
   |
User question --> embed --> find most similar chunks
   |
   v
[5] Question + top chunks --> LLM
   |
   v
[6] Answer + page citations
```

5. Technology Stack

Layer	Suggested Tools
Language	Python 3.10+
PDF parsing	PyMuPDF or pypdf
Orchestration	LangChain or LlamaIndex
Embeddings	sentence-transformers (local) or an embeddings API

Layer	Suggested Tools
Vector database	FAISS or ChromaDB
LLM	Any LLM API, or a local model through Ollama
Interface	Streamlit or Gradio

6. Project Structure

```
docuchat/
|-- app.py            # Streamlit / Gradio user interface
|-- ingest.py         # PDF loading, chunking, embedding
|-- retriever.py      # Vector search
|-- chat.py           # Prompt building, LLM call, citations
|-- requirements.txt
|-- data/             # Uploaded PDFs
|-- vectorstore/      # Saved index
|-- README.md
```

7. Installation and Usage

```
git clone https://github.com/<your-username>/docuchat.git
cd docuchat
python -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate
pip install -r requirements.txt
export LLM_API_KEY="your_key_here"   # only if using an API
streamlit run app.py
```

- Open the app in your browser and upload your PDF files.
- Wait for indexing to finish.
- Ask a question and read the answer with its citations.

8. Example Interaction

Question	Answer
What were the main findings of the study?	Participants who followed the program improved by 23% over the control group. (Source: research_paper.pdf, p. 7)
What is the author's favorite color?	I couldn't find this in your documents.

9. Limitations

- Scanned PDFs (images) need OCR before they can be searched.
- Tables, charts, and equations may not be extracted accurately.
- Answer quality depends on chunk size, retrieval settings, and the chosen LLM.
- Very large document collections may need a more scalable vector database.

10. Future Scope

Planned Improvement	Benefit
OCR for scanned PDFs	Search image-based documents
Highlight cited passage in the PDF viewer	Faster verification of answers
Support DOCX, TXT, and web pages	Wider range of sources
Chat history and follow-up questions	More natural conversations

Planned Improvement	Benefit
Multilingual support, including Tamil	Reach more users
Docker deployment	Easier setup and hosting
Evaluation suite	Measure and improve answer accuracy

11. Conclusion

DocuChat shows how RAG can make AI answers trustworthy by tying every response to its source. It is a practical, beginner-friendly project that combines document processing, embeddings, vector search, and LLMs into one useful tool, with plenty of room to grow.